SYSTEM ARCHITECTURE DOCUMENT

Lane Keep Assist (LKA) System

Team Equinox

aBAJA SAEINDIA 2026 — Software Virtual World Simulation

1. System Overview

This document describes the software architecture of the Lane Keep Assist (LKA) system developed by Team Equinox for the aBAJA SAEINDIA 2026 Software Virtual World Simulation event. The system operates within IPG CarMaker 15.0 and interfaces via the CarMaker Python APO (Asynchronous Process Object). The perception stack relies exclusively on a Camera RSI sensor; the system does not utilise ground-truth lane data or DVA-based lane quantities.

The architecture was designed for modularity, testability, and real-time performance at a 30 Hz control loop rate, with all five Python modules following a strict downward dependency hierarchy.

Parameter	Value
Simulator	IPG CarMaker 15.0
Rendering	MovieNX
Camera Sensor	Camera RSI — 640×480 RGB, ~30 Hz over TCP port 2210
Target Speed	30 km/h
Control Loop Rate	30 Hz
Vehicle Model	Demo_IPG_CompanyCar (wheelbase 2.622 m, mass 1,391 kg)
Language	Python 3 with OpenCV, NumPy, IPG CarMaker Python API

2. Module Architecture

The architecture is decomposed into five specialised Python modules. Following a strict downward dependency flow, the main loop orchestrates execution while sub-modules remain decoupled to ensure modularity and ease of unit testing. No sub-module imports the main loop.

Module	Role	Key Outputs
Equinox_Main.py	Main control loop — bridges CarMaker and the perception/control stack. Initialises APO, acquires camera frames, applies Pure Pursuit + PID, and writes steering commands.	Steering command (rad)
perception_engine.py	Computer vision module. Applies BEV perspective warp, Sliding Window lane search, polynomial fitting, and computes lateral error and lookahead offset.	Lateral error (m), lookahead offset (m)
equinox_config.py	Single source of truth for all tunable parameters: camera resolution, BEV trapezoid coordinates, pixels-per-metre, PID gains, and safety thresholds.	Constants (imported)

equinox_utils.py	Utility module: RMSE accumulator, DNF watchdog (triggers if front axle leaves lane for >5 s), and CSV telemetry logging.	RMSE value, DNF flag, lka_run.csv
tcp_client.py	TCP socket client that connects to the Camera RSI sensor stream on localhost:2210. Handles frame decoding, buffering, and automatic reconnection.	Raw BGR frame (NumPy array)

3. Data Flow

Each control cycle (nominally 1/30 s) executes the following pipeline sequentially. The pipeline is designed to be stateless where possible; persistent state (polynomial fit cache, RMSE accumulator) is explicitly managed in dedicated modules.

Step	Module	Description
1	**tcp_client**	Receive raw 640×480 RGB frame from Camera RSI over TCP
2	**perception_engine**	Convert to grayscale → apply binary thresholding
3	**perception_engine**	Warp to Bird's-Eye View (source trapezoid → top-down rectangle)
4	**perception_engine**	Bottom-half histogram → find left/right lane base positions
5	**perception_engine**	Sliding window search upward → collect lane pixel coordinates
6	**perception_engine**	Fit 1st-order polynomial to each lane → compute lane centre
7	**perception_engine**	Compute lateral error = lane centre − image centre (metric)
8	**perception_engine**	Compute lookahead offset at 5.0 m ahead on fitted lane
9	**Equinox_Main**	Pure Pursuit: compute baseline steering angle from lookahead geometry
10	**Equinox_Main**	PID: compute correction term from lateral error
11	**Equinox_Main**	Sum and rate-limit → clamp to ±0.20 rad road-wheel angle
12	**Equinox_Main**	Write road-wheel angle to CarMaker APO (DVA write)
13	**equinox_utils**	Log telemetry to CSV; update RMSE accumulator; check DNF watchdog

4. Perception Pipeline — BEV + Sliding Window

The Team Equinox perception approach utilises a classical computer vision pipeline. After evaluating high-complexity alternatives including CLAHE + Percentile Threshold, Hough Transform, Adaptive Thresholding, and Frangi Ridge Detection — all of which failed due to near-black camera frames in the default CarMaker exposure configuration — the BEV + Sliding Window method was selected for its reliability in varying CarMaker light conditions. Camera settings were corrected to ISO 800, F/2.8 with Filmic tone mapping to resolve the exposure issue.

4.1 Bird's-Eye View Transform

A perspective warp maps a trapezoidal region of the front-facing camera image to a top-down rectangle. This eliminates perspective foreshortening, ensuring lane lines appear as near-parallel vertical strips regardless of their distance from the camera. The pixels-per-metre calibration ratio converts pixel offsets to metric lateral errors, decoupling the perception output from image resolution.

4.2 Sliding Window Lane Search

The algorithm utilises a bottom-half column histogram to identify lane origins. A stack of 9 windows (±50 px margin) searches upward, re-centring on the centroid of detected white pixels at each step. This method allows the system to bridge gaps in dashed lines and track sharp curves effectively without requiring explicit angle assumptions about lane orientation.

4.3 Polynomial Fitting and Gap Bridging

A first-order polynomial is fitted to detected pixels within each window stack. The lane centre and lookahead point are derived from the mean of the two fitted curves. To handle intermittent occlusions or lighting glares, the system caches the last 15 frames of valid polynomial fits, allowing the vehicle to maintain stable steering through brief sensor dropouts without losing control stability.

5. Control Architecture — Pure Pursuit + PID

Team Equinox utilises a hybrid controller combining Pure Pursuit geometry and PID feedback. A standalone PID controller reacts to lateral error but has no knowledge of vehicle geometry or road curvature, making it prone to oscillation on curves. The Pure Pursuit component provides a geometrically grounded baseline: it computes the exact steering angle required to follow an arc from the vehicle's current position to a lookahead point on the detected lane, using the vehicle wheelbase and steering ratio. The PID term then compensates for residual steady-state error.

Parameter	Value	Rationale
Lookahead distance	5.0 m	Provides ~0.6 s preview at 30 km/h; balances responsiveness and smoothness.
Vehicle wheelbase	2.622 m	Matched to Demo_IPG_CompanyCar specification.
Steering ratio	16:1	Vehicle model parameter — maps road-wheel angle to virtual steering wheel.
Max road-wheel angle	±0.20 rad	Physical steering limit enforced in software to prevent spin-out.
Rate limit	0.03 rad/frame	Prevents actuator saturation and jerky motion; maintains tire grip.

PID (Kp / Ki / Kd)	0.10 / 0.005 / 0.08	Tuned empirically; Ki eliminates steady-state offset on straightaways; Kd damps overshoot on transitions.

6. Safety Mechanisms

Five independent safety mechanisms ensure the vehicle remains within the lane boundaries and the control system remains within safe actuator limits throughout each run.

Mechanism	Implementation	Behaviour
DNF Watchdog	equinox_utils.py	If the front axle exits lane boundaries for >5 consecutive seconds, the watchdog logs a DNF event and halts the run to comply with competition rules.
Steering Rate Limiter	Equinox_Main.py	Output delta is capped at 0.03 rad per frame regardless of commanded value, preventing actuator saturation and maintaining tire grip during sudden lane corrections.
Angle Saturation Clamp	Equinox_Main.py	Final road-wheel command is strictly clamped to ±0.20 rad, matching the physical steering limit of the Demo_IPG_CompanyCar model.
Fit Cache Fallback	perception_engine.py	Uses the last valid polynomial fit for up to 15 frames if no lanes are detected, preventing zero-steering commands during dashed line gaps or partial occlusion.
TCP Reconnect	tcp_client.py	Automatically reconnects to the Camera RSI stream on socket drop with exponential backoff, ensuring continuity of the camera feed without manual intervention.

7. Logged Outputs

All run outputs are written to results/EQUINOX_LKA/. The logging subsystem is managed by equinox_utils.py and runs asynchronously to avoid introducing latency into the 30 Hz control loop.

File / Folder	Description
Data/Logs/lka_run.csv	Per-frame telemetry: timestamp, lateral error, steering angle, speed, yaw rate.
debug_frames/	Per-frame debug images (BEV view, sliding window visualisation) — archived post-run for offline analysis.
debug_raw_frame.png	The final raw camera frame received from CarMaker during the run.

debug_bev.png	The last Bird's-Eye View frame after perspective warp.
debug_vision.png	Last annotated front-view frame with lane centre overlay, lateral error metric, and sliding window visualisation.